

WEEK-4

Session7,8

TASK-1:

Process Synchronization and Child Process Monitoring Using waitpid()

CODE:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

int main()
{
    pid_t child1, child2;

    int status;

    printf("===== PROCESS SYNCHRONIZATION USING waitpid() =====\n");

    child1 = fork();

    if (child1 < 0)
    {
        perror("fork failed");

        return 1;
    }

    if (child1 == 0)
    {
        printf("\nChild 1 started. PID = %d\n", getpid());

        printf("Child 1 is working...\n");

        sleep(3);

        printf("Child 1 completed.\n");

        exit(10);
    }

    child2 = fork();
```

```

if (child2 < 0)
{
    perror("fork failed");
    return 1;
}
if (child2 == 0)
{
    printf("\nChild 2 started. PID = %d\n", getpid());
    printf("Child 2 is working...\n");
    sleep(5);
    printf("Child 2 completed.\n");
    exit(20);
}
printf("\nParent process. PID = %d\n", getpid());
printf("Parent is waiting for Child 1...\n");
if (waitpid(child1, &status, 0) == -1)
{
    perror("waitpid failed");
    return 1;
}
if (WIFEXITED(status))
{
    printf("Child 1 terminated with exit status = %d\n",
        WEXITSTATUS(status));
}
printf("\nParent is waiting for Child 2...\n");
if (waitpid(child2, &status, 0) == -1)
{
    perror("waitpid failed");
}

```

```

        return 1;
    }
    if (WIFEXITED(status))
    {
        printf("Child 2 terminated with exit status = %d\n",
               WEXITSTATUS(status));
    }
    printf("\nAll child processes have completed.\n");
    printf("Parent process exiting.\n");
    return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# nano waitpid_demo.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g waitpid_demo.c -o waitpid_demo
root@LAPTOP-RJFD8EBT:~# ./waitpid_demo
===== PROCESS SYNCHRONIZATION USING waitpid() =====

Child 1 started. PID = 2522
Child 1 is working...

Parent process. PID = 2521
Parent is waiting for Child 1...

Child 2 started. PID = 2523
Child 2 is working...
Child 1 completed.
Child 1 terminated with exit status = 10

Parent is waiting for Child 2...
Child 2 completed.
Child 2 terminated with exit status = 20

All child processes have completed.

```

TASK-2:

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

CODE:

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/stat.h>

```

```

#define MAX_PATH_LENGTH 4096

int main(int argc, char *argv[]){
    char *path;
    char *path_copy;
    char *directory;
    char full_path[MAX_PATH_LENGTH];
    struct stat file_info;

    if (argc != 2)
    {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    path = getenv("PATH");
    if (path == NULL)
    {
        printf("PATH environment variable is not set.\n");
        return 1;
    }

    printf("==== PATH COMMAND RESOLVER =====\n");
    printf("Command to find: %s\n", argv[1]);
    printf("\nPATH variable:\n%s\n", path);
    path_copy = strdup(path);
    if (path_copy == NULL)
    {
        perror("strdup");
        return 1;
    }

    printf("\nSearching directories:\n");
    directory = strtok(path_copy, ":");

```

```

while (directory != NULL) {
    printf("Checking: %s\n", directory);
    snprintf(full_path, sizeof(full_path), "%s/%s",
        directory, argv[1]);
    if (stat(full_path, &file_info) == 0)
    {
        printf(" Found: %s\n", full_path);
        if (S_ISREG(file_info.st_mode))
        {
            if (access(full_path, X_OK) == 0)
            {
                printf(" Executable: YES\n");
                printf(" Execute permission: YES\n");
                free(path_copy);
                printf("\nCommand resolved successfully.\n");
                printf("Executable path: %s\n", full_path);
                return 0;
            } else{
                printf(" Execute permission: NO\n");
            }
        } else{
            printf(" Regular file: NO\n");
        }
    }
    directory = strtok(NULL, ":");
}

free(path_copy);

printf("\nCommand '%s' was not found in PATH.\n", argv[1]);
printf("Command resolution failed.\n");

```

}

TASK-3:

CODE:

```
#include <stdlib.h>
```

```
#include <ctype.h>
```

```
void expand_variables(const char *input, char *output, size_t output_size)
```

{

```
size_t j = 0;
```

{

{

```
size_t k = 0;
```

```

const char *value;

i++;

if (input[i] == '{')
{
    i++;

    while (input[i] != '\0' &&
           input[i] != '}' &&
           k < sizeof(var_name) - 1 {
        var_name[k++] = input[i++];
    }

    var_name[k] = '\0';

    if (input[i] == '}')
        i++;
}
else {
    while ((isalnum((unsigned char)input[i]) ||
           input[i] == '_' ) &&
           k < sizeof(var_name) - 1) {
        var_name[k++] = input[i++];
    }

    var_name[k] = '\0';
}

if (k == 0) {
    output[j++] = '$';

    continue;
}

value = getenv(var_name);

if (value != NULL){
    size_t len = strlen(value);

    if (j + len >= output_size)

```

```

        len = output_size - j - 1;
        memcpy(output + j, value, len);
        j += len;
    }else {
        printf("Undefined variable: %s\n", var_name);
    }
} else {
    output[j++] = input[i++];
}
}
output[j] = '\0';
}

void expand_nested(const char *input, char *output, size_t output_size)
{
    char current[MAX_OUTPUT];
    char next[MAX_OUTPUT];
    strncpy(current, input, sizeof(current) - 1);
    current[sizeof(current) - 1] = '\0';
    for (int i = 0; i < 5; i++)
    {
        expand_variables(current, next, sizeof(next));
        if (strcmp(current, next) == 0)
            break;
        strncpy(current, next, sizeof(current) - 1);
        current[sizeof(current) - 1] = '\0';
    }
    strncpy(output, current, output_size - 1);
    output[output_size - 1] = '\0';
}

```

```
int main(){
    char input[MAX_OUTPUT];
    char output[MAX_OUTPUT];

    printf("===== VARIABLE EXPANSION DEMONSTRATION =====\n");
    setenv("USER_NAME", "Rajini", 1);
    setenv("COLLEGE", "KLEF", 1);
    setenv("COURSE", "CSIT", 1);
    setenv("STUDENT", "$USER_NAME", 1);
    printf("\nEnvironment variables set for testing:\n");
    printf("USER_NAME = %s\n", getenv("USER_NAME"));
    printf("COLLEGE  = %s\n", getenv("COLLEGE"));
    printf("COURSE   = %s\n", getenv("COURSE"));
    printf("STUDENT  = %s\n", getenv("STUDENT"));
    strcpy(input, "Hello $USER_RAJINI");
    expand_nested(input, output, sizeof(output));
    printf("\nTest 1:\n");
    printf("Input : %s\n", input);
    printf("Output: %s\n", output);
    strcpy(input, "College: ${COLLEGE}, Course: $COURSE");
    expand_nested(input, output, sizeof(output));
    printf("\nTest 2:\n");
    printf("Input : %s\n", input);
    printf("Output: %s\n", output);
    strcpy(input, "Value: $UNDEFINED_VAR");
    expand_nested(input, output, sizeof(output));
    printf("\nTest 3 - Undefined variable:\n");
    printf("Input : %s\n", input);
    printf("Output: %s\n", output);
    strcpy(input, "Student: $STUDENT");
```

```

    expand_nested(input, output, sizeof(output));

    printf("\nTest 4 - Nested variable:\n");

    printf("Input : %s\n", input);

    printf("Output: %s\n", output);

    strcpy(input, "$USER_RAJINI studies $COURSE at $COLLEGE");

    expand_nested(input, output, sizeof(output));

    printf("\nTest 5 - Multiple variables:\n");

    printf("Input : %s\n", input);

    printf("Output: %s\n", output);

    printf("\nVariable expansion testing completed successfully.\n");

    return 0;
}

```

```

root@LAPTOP-RJFD8EBT:~# nano variable_expand.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g variable_expand.c -o variable_expand
root@LAPTOP-RJFD8EBT:~# ./variable_expand
===== VARIABLE EXPANSION DEMONSTRATION =====

Environment variables set for testing:
USER_NAME = Rajini
COLLEGE    = KLH
COURSE     = CSIT
STUDENT    = $USER_NAME
Undefined variable: USER_RAJINI

Test 1:
Input : Hello $USER_RAJINI
Output: Hello

Test 2:
Input : College: ${COLLEGE}, Course: $COURSE
Output: College: KLH, Course: CSIT
Undefined variable: UNDEFINED_VAR

Test 3 - Undefined variable:
Input : Value: $UNDEFINED_VAR
Output: Value:

Test 4 - Nested variable:
Input : Student: $STUDENT
Output: Student: Rajini
Undefined variable: USER_RAJINI

Test 5 - Multiple variables:
Input : $USER_RAJINI studies $COURSE at $COLLEGE
Output: studies CSIT at KLH

Variable expansion testing completed successfully.
root@LAPTOP-RJFD8EBT:~#

```

TASK-4:

To Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

CODE: #include <stdio.h>

#include <stdlib.h>

```

#include <string.h>
#include <unistd.h>
#include <limits.h>

#define INPUT_SIZE 256
#define MAX_ARGS 20

/* Function pointer for built-in commands */
typedef int (*builtin_func)(char **args);

/* cd command */
int builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        printf("cd: missing argument\n");
        return 1;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
        return 1;
    }

    return 0;
}

/* pwd command */

```

```
int builtin_pwd(char **args)
{
    char cwd[PATH_MAX];

    (void)args;

    if (getcwd(cwd, sizeof(cwd)) == NULL)
    {
        perror("pwd");
        return 1;
    }

    printf("%s\n", cwd);

    return 0;
}
```

```
/* echo command */
int builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");
    }
}
```

```

        i++;
    }

    printf("\n");

    return 0;
}

/* export command */
int builtin_export(char **args)
{
    char *equal_sign;

    if (args[1] == NULL)
    {
        printf("Usage: export NAME=VALUE\n");
        return 1;
    }

    equal_sign = strchr(args[1], '=');

    if (equal_sign == NULL)
    {
        printf("Usage: export NAME=VALUE\n");
        return 1;
    }

    *equal_sign = '\0';

```

```

if (setenv(args[1], equal_sign + 1, 1) != 0)
{
    perror("export");
    return 1;
}

printf("Exported %s=%s\n", args[1], equal_sign + 1);

return 0;
}

/* help command */
int builtin_help(char **args)
{
    (void)args;

    printf("\n===== BUILT-IN COMMANDS =====\n");
    printf("cd <directory> - Change current directory\n");
    printf("pwd          - Display current directory\n");
    printf("echo <text>    - Display text\n");
    printf("export N=V     - Set environment variable\n");
    printf("help          - Display available commands\n");
    printf("exit          - Exit the shell\n");

    return 0;
}

/* exit command */
int builtin_exit(char **args)

```

```
{
    (void)args;

    printf("Exiting built-in shell...\n");

    exit(0);
}

/* Dispatch table */
struct builtin
{
    const char *name;
    builtin_func function;
};

struct builtin dispatch_table[] =
{
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"echo", builtin_echo},
    {"export", builtin_export},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

int dispatch_count =
    sizeof(dispatch_table) / sizeof(dispatch_table[0]);

/* Execute built-in command */
```

```

int execute_builtin(char **args)
{
    int i;

    for (i = 0; i < dispatch_count; i++)
    {
        if (strcmp(args[0], dispatch_table[i].name) == 0)
        {
            return dispatch_table[i].function(args);
        }
    }

    printf("Invalid command: %s\n", args[0]);

    return 1;
}

```

```

/* Parse command into tokens */
int parse_input(char *input, char **args)
{
    int count = 0;
    char *token;

    token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count] = token;
        count++;
    }
}

```

```

        token = strtok(NULL, " \\t\\n");
    }
    args[count] = NULL;
    return count;
}

int main(void)
{
    char input[INPUT_SIZE];
    char *args[MAX_ARGS];
    int argc;

    printf("==== BUILT-IN COMMAND DISPATCH TABLE ====\\n");
    printf("Type 'help' to see available commands.\\n");
    printf("Type 'exit' to quit.\\n");
    while (1)
    {
        printf("\\nbuiltin-shell> ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }
        argc = parse_input(input, args);
        if (argc == 0)
        {
            continue;
        }
        execute_builtin(args);
    }
}

```

```
return 0;
}
```

```
root@LAPTOP-RJFD8EBT:~# nano builtin_shell.c
root@LAPTOP-RJFD8EBT:~# gcc -Wall -Wextra -g builtin_shell.c -o builtin_shell
root@LAPTOP-RJFD8EBT:~# ./builtin_shell
===== BUILT-IN COMMAND DISPATCH TABLE =====
Type 'help' to see available commands.
Type 'exit' to quit.

builtin-shell> help

===== BUILT-IN COMMANDS =====
cd <directory> - Change current directory
pwd            - Display current directory
echo <text>    - Display text
export N=V     - Set environment variable
help          - Display available commands
exit          - Exit the shell

builtin-shell> pwd
/root

builtin-shell> echo hello
hello

builtin-shell> cd/temp
Invalid command: cd/temp

builtin-shell> hello123
Invalid command: hello123

builtin-shell> exit
Exiting built-in shell...
S root@LAPTOP-RJFD8EBT:~# |
```